



EST160 - Aula 06

Lucas Sousa

15 de Setembro de 2026

Conteúdos

Estratégias de Imputação

Limpeza de Textos e Variáveis Categóricas

Normalização e Padronização de Variáveis

Estratégias de Imputação

Agora que sabemos que podemos ter dados faltantes e que podemos explorar esses dados, precisamos procurar estratégias para lidar com essa falta.

Nossas opções são:

- ▶ **Imputação:** onde valores são preenchidos no lugar dos dados faltantes;
- ▶ **Omissão:** onde amostras com dados inválidos ou variáveis são descartadas;
- ▶ **Análise:** onde aplicamos métodos que não sejam afetados pelos dados faltantes.

Estratégias de Imputação

Um caso onde poderíamos escolher a **omissão** para realizar a análise seria o caso Dados Faltantes Completamente Aleatórios (MCAR), mas é raro estarmos nesse caso;

Estratégias de Imputação

Um caso onde poderíamos escolher a **omissão** para realizar a análise seria o caso Dados Faltantes Completamente Aleatórios (MCAR), mas é raro estarmos nesse caso;

MAR será uma suposição usual de modelos de imputação, mas não podemos assumir isso sem tomar muito cuidado;

Estratégias de Imputação

Um caso onde poderíamos escolher a **omissão** para realizar a análise seria o caso Dados Faltantes Completamente Aleatórios (MCAR), mas é raro estarmos nesse caso;

MAR será uma suposição usual de modelos de imputação, mas não podemos assumir isso sem tomar muito cuidado;

Além da nossa análise descritiva e exploratória, é importante sabermos de onde os dados vieram e como foram coletados, porque isso pode adicionar informação para nossas decisões.

Van Buuren (2012) é uma boa referência para o assunto. Little and Rubin (2019) é uma referência para um futuro mais técnico.

Estratégias de Imputação

Remoção dos Dados Falantes (Listwise deletion)

Eliminar as **observações** faltantes tem a vantagem da **conveniência**.

Se os dados forem MCAR, a remoção produz estimadores não viesados para média, variância e coeficientes de regressão.

Se os dados são MCAR, a remoção produz erros padrões e níveis de significância que são corretos para o conjunto dos dados reduzidos, mas são geralmente maiores em relação aos dados completos.

Estratégias de Imputação

Remoção dos Dados Falantes

A desvantagem de remover os NAs é que isso é potencialmente um desperdício;

Se o número de variáveis é grande, então não é incomum nos livramos de de mais da metade dos dados originais;

Uma amostra muito menor pode reduzir a nossa capacidade de detectar efeitos de interesse.

Estratégias de Imputação

Remoção dos Dados Falantes

Outras questões

- ▶ Se os dados não forem MCAR, deletar as observações pode provocar muito viés na média e coeficientes de regressão;
- ▶ Apagar as observações pode trazer inconsistências nos relatórios, visto que podemos querer criar subtabelas com diferentes subconjuntos dos dados.

Estratégias de Imputação

Remoção em Pares (Pairwise deletion)

Também conhecida como *available-case analysis* (análise de casos disponíveis), é uma forma de tentar mitigar os problemas da *listwise deletion*

Nesse caso, calculamos as médias e (co)variâncias de todos os dados observados

- A média da variável X é baseada em todos os casos observados nos dados da própria X ;
- A média da variável Y é baseada em todos os casos observados nos dados da própria Y • Para correlação e covariância, são tomados os dados onde X e Y não possuem faltas simultâneas

A partir daí, utilizamos essas informações para realizar alguma análise estatística.

Estratégias de Imputação

Remoção em Pares (Pairwise deletion)

Esse método é simples e parece usar toda informação disponível, o que reduz a questão do desperdício

Se temos dados MCAR, ele produz estimadores consistentes para média, correlação e covariância de acordo com Little and Rubin (2019)

Novamente os estimadores podem ser viesados se os dados não são MCAR

As matrizes de covariância e correlação podem não ser positivas definidas (uma propriedade importantes para muitos procedimentos multivariados que veremos no futuro)

Estratégias de Imputação

Remoção em Pares (Pairwise deletion)

Se os dados forem muito correlacionados, os problemas podem ser mais severos e temos uma questão com o tamanho da amostra e o erro padrão;

Precisamos assumir normalidade aproximada dos dados (outro conceito que veremos no futuro), mas geralmente temos tipos mistos de variáveis

Estratégias de Imputação

Remoção em Pares (Pairwise deletion)

É interessante utilizar toda informação disponível, mas uma análise apropriada da *pairwise matrix* requer técnicas de otimização sofisticadas e fórmulas especiais para calcular erros padrões, o que tira o ponto de sua utilidade (em relação a praticidade)

Esse método funciona melhor se os dados seguem aproximadamente uma normal multivariada, se a correção entre as variáveis é baixa e se a suposição de MCAR é plausível, do contrário não é recomendado.

Estratégias de Imputação

Imputação Pela Média

Uma forma rápida de lidar com os dados faltantes é imputando pela média (ou moda, se os dados forem categóricos)

A imputação pela média distorce a distribuição dos dados de várias formas

Podemos acabar criando coisas como uma distribuição bimodal, o erro padrão pode ser artificialmente reduzido pela média

Estratégias de Imputação

Imputação Pela Média

Esse método é rápido e simples, mas pode subestimar a variância e distorcer relações entre as variáveis

Também irá viesar quase qualquer outro estimador que não seja a própria média, e ainda tratá viés para a média se os dados não forem MCAR

Esse método deve ser utilizado somente como um conserto rápido quando a quantidade de dados faltantes é baixa, e deve ser evitado no geral

Estratégias de Imputação

Imputação Pela Mediana

Os problemas ao utilizar a mediana serão semelhante, mas temos a vantagem dessa medida sofrer menos com a presença de outliers, e podemos considerá-la no lugar da média com os mesmos cuidados anteriores

Se temos distribuições assimétricas, pode ser um caso a se considerar no lugar da média, principalmente se temos uma quantidade menor de dados faltantes

Estratégias de Imputação

Imputação Pela Moda

Para a moda, as vantagens da praticidade serão parecidas, mas também podemos criar diversos problemas em relação a variabilidade e relação das variáveis

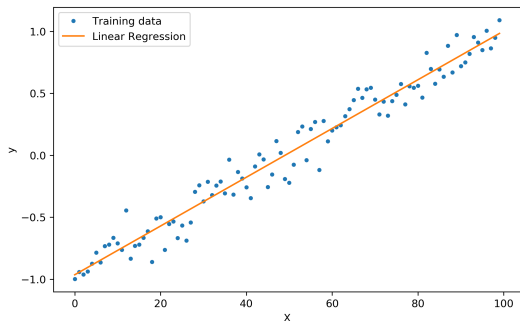
Se temos uma categoria muito dominante, então pode ser razoável utilizá-la, porém precisamos tomar cuidado se os dados forem MAR, pois estamos justamente ignorando a relação entre as variáveis

Podemos preferir transformar os NAs em fatores, criando categorias para os dados faltantes, o que evita a exclusão de variáveis

Estratégias de Imputação

Imputação Por Regressão

Regressão é uma técnica comum para estimativa variáveis dependentes



Nesse caso, temos uma equação onde $\mathbf{Y} = \beta_0 + \beta_1 \mathbf{X} + \epsilon$

Queremos encontrar β_0 e β_1 que melhor se ajustam ao modelo, isto é, que melhor aproximam nossos dados.

Estratégias de Imputação

Imputação Por Regressão

Imputação por regressão incorpora informação sobre outras variáveis com a ideia de produzir uma imputação mais inteligente

Nesse caso, a variável que queremos imputar será ajustada como variável dependente de um modelo de regressão que consideramos apropriado

Encontrar o modelo depende do tipo de dados. Podemos por exemplo ter uma relação linear entre as variáveis, mas a relação pode ser quadrática, exponencial, etc.

Estratégias de Imputação

Imputação Por Regressão

Uma questão a ser observada é que podemos ter viés na correlação das nossas variáveis, principalmente se o número de variáveis faltantes aumentar;

Imputação por regressão nos leva a uma estimação não viesada da média sob MCAR.

Se as variáveis explanatórias (as que estamos utilizando para prever a que falta) estiverem completas, isso também nos dá uma estimativa não viesada dos betas β

Além disso, os coeficiente de regressão são não viesados sob a suposição de MAR se os fatores que influenciam a falta são parte do nosso modelo de regressão

Estratégias de Imputação

Imputação Por Regressão

Muita atenção deve ser tomada quando lidamos com esse tipo de imputação. A imputação vai ser mais realista quando a informação faltante está na verdade codificada de outra forma

Van Buuren (2012) destaca que esse é um tipo perigoso de imputação porque pode nos levar a acreditar que temos uma boa previsão

Esse método fortalece artificialmente a relação dos dados e envies a correlação (aumenta). Além disso, a Variabilidade é subestimada

Podemos ter muitos falso positivos e relações espúrias.

Estratégias de Imputação

Last Observation Carried Forward (LOCF) and Baseline Observation Carried Forward (BOCF)

Esses métodos são comuns para dados **longitudinais**, onde temos várias observações do mesmo indivíduo.

LOCF utiliza a última observação do indivíduo e BOCF utiliza a *baseline*, isto é, a primeira.

Por exemplo, podemos querer avaliar um tratamento médico por meses. Com isso, teríamos que ter diversas observações a respeito da mesma pessoa.

Estratégias de Imputação

Last Observation Carried Forward (LOCF) and Baseline Observation Carried Forward (BOCF)

O LOCF é conveniente porque gera uma base de dados completa

Pode ser aplicado com confiança nos casos onde temos certeza que os dados faltantes deveriamos ser administrados como dados longitudinais

Mas LOCF pode gerar viés mesmo sob MCAR, que é a melhor suposição.

Ambos são muito restritos, mas se não queremos supor que um paciente continuou melhorando, por exemplo, utilizaos o BOCF. Não é recomendado que sejam utilizados como primeira abordagem.

Estratégias de Imputação

Estratégias mais avançadas:

- ▶ Multivariate Imputation by Chained Equations (MICE)
- ▶ K-Nearest Neighbors (KNN) Imputation
- ▶ MissForest/Decision Trees
- ▶ Support Vector Machines (SVM)

Limpeza de Textos e Variáveis Categorias

Agora vamos começar a focar um pouco mais na limpeza dos nossos dados.

Vimos inicialmente como realizar a limpeza dos dados com as funções básicas do R, mas agora vamos utilizar os pacotes `{stringr}` e `{forcats}` para relializar nossas tarefas.



Limpeza de Textos

Para criar textos, já vimos que podemos utilizar aspas ou apóstrofes.

Não há diferença prática, mas o padrão do tidyverse é utilizar aspas, a não ser que haja aspas no seu próprio texto.

```
string1 <- "This is a string"
```

```
string2 <- 'If I want to include a "quote"  
  inside a string, I use single quotes'
```

```
> string2
```

```
[1] "If I want to include a \"quote\" inside a  
  string, I use single quotes"
```

Limpeza de Textos

Para criar textos, já vimos que podemos utilizar aspas ou apóstrofes.

Não há diferença prática, mas o padrão do tidyverse é utilizar aspas, a não ser que haja aspas no seu próprio texto.

```
string1 <- "This is a string"
```

```
string2 <- 'If I want to include a "quote"  
  inside a string, I use single quotes'
```

```
> string2
```

```
[1] "If I want to include a \"quote\" inside a  
  string, I use single quotes"
```

```
> string2 <- "If I want to include a "quote"  
  inside a string, I use single quotes"
```

```
Erro: unexpected symbol em "string2 <- "If I  
  want to include a "quote"
```

Limpeza de Textos

Em linguagens que possuam caracteres especiais, é comum que tenhamos uma forma de escrever aquele caractere como string (*scape*). No R, podemos fazer isso com o símbolo `\` antes do caractere que queremos utilizar.

```
double_quote <- "\" # or '"'  
single_quote <- "'" # or '"'  
backslash <- "\\"
```

Limpeza de Textos

Em linguagens que possuam caracteres especiais, é comum que tenhamos uma forma de escrever aquele caractere como string (*scape*). No R, podemos fazer isso com o símbolo `\` antes do caractere que queremos utilizar.

```
double_quote <- "\" # or ''  
single_quote <- '\",' # or ""  
backslash <- "\\"
```

Para visualizar o conteúdo do string sem o mostrar o scape, podemos utilizar `str_view()` ou `writeLines()`, nativa do R.

```
> x <- c(single_quote, double_quote, backslash)  
> x  
[1] ",'" "\" "\\"
```

```
> str_view(x)  
[1] | '  
[2] | "  
[3] | \
```

Síndrome do Palito de Dente (*Toothpick Syndrome*)

Se quisermos escrever muitos caracteres especiais como strings, é comum que o código fique confuso.

```
tricky <- "double_quote <- \"\\\"\" # or '\",'
single_quote <- '\\', # or \"'\"

str_view(tricky)
#> [1] | double_quote <- "\"\" # or ', '
#>
| single_quote <- '\\', # or \"', \"
```

Síndrome do Palito de Dente (*Toothpick Syndrome*)

Para lidar com isso, utilizamos *raw strings*, utilizando `r"()"`.

```
tricky <- r"(double_quote <- "\" # or '",'  
single_quote <- '\',' # or "'")"
```

```
str_view(tricky)
```

```
#> [1] | double_quote <- "\" # or '",'
```

```
#> | single_quote <- '\',' # or "'"
```

- ▶ Se tivermos parênteses () no nosso texto, podmos utilizar `r"[]"` ou `r"{}"`
- ▶ Se isso não for suficiente, podemos utilizar algo do tipo `r"-( )-"` para modificar o que fecha o texto.

Caracteres Especiais

Geralmente temos códigos para acessar caracteres especiais de outras línguas, ou até mesmo emojis.

```
x <- c("one\ntwo", "one\ttwo", "\u00b5", "\nU0001f604")
```

```
> str_view(x)
[1] | one
   | two
[2] | one{\t}two
[3] | μ
[4] | aqui deveria ter saído um emoji
```

Utilize ?Quotes para acessar os caracteres especiais.

Funções do pacote {stringr}

A função `str_c()` toma qualquer número de número de vetores como argumentos e retorna um vetor de caracteres.

```
str_c("x", "y")  
#> [1] "xy"  
str_c("x", "y", "z")  
#> [1] "xyz"  
str_c("Hello ", c("John", "Susan"))  
#> [1] "Hello John" "Hello Susan"
```

Essa função é similar a função `paste0()` nativa do R, mas ela foi pensada para ser usada com as funções do tidyverse

str_c()

```
> df <- tibble(name = c("Flora", "David", "Terra",  
  " ", NA))  
> df |> mutate(greeting = str_c("Hi ", name, "!"  
  ))
```

```
# A tibble: 4 × 2  
  name    greeting  
  <chr>   <chr>  
1 Flora Hi  Flora!  
2 David Hi  David!  
3 Terra Hi  Terra!  
4 NA     NA
```

str_c()

Para fazer os NAs aparecerem, podemos utilizar a função `coalesce()`

```
df |>
  mutate(
    greeting1 = str_c("Hi ", coalesce(name,
    "you"), "!"),
    greeting2 = coalesce(str_c("Hi ", name,
    "!"), "Hi!")
  )

# A tibble: 4 × 3
  name   greeting1 greeting2
<chr> <chr>      <chr>
1 Flora Hi Flora! Hi Flora!
2 David Hi David! Hi David!
3 Terra Hi Terra! Hi Terra!
4 NA    Hi you!    Hi!
```

str_glue()

A função `str_glue()` (ou `glue()` se tiver sendo utilizado o pacote `{glue}`) nos ajuda a fazer a junção dos strings sem ter que escrever tantas aspas.

Nese caso, tudo que tiver dentro de chaves `{}` será interpretado como se tivesse fora das aspas.

```
> df |> mutate(greeting = str_glue("Hi {name}!"))
```

```
# A tibble: 4 × 2
  name    greeting
  <chr>   <glue>
1 Flora Hi Flora!
2 David Hi David!
3 Terra Hi Terra!
4 NA     Hi NA!
```

str_glue()

Se precisarmos escrever chaves no texto, o scape que vamos utilizar é chaves duplas `{{`.

```
df |> mutate(greeting = str_glue "{{Hi {name}!}}")
```

```
# A tibble: 4 × 2  
  name    greeting  
  <chr> <glue>  
1 Flora {Hi Flora!}  
2 David {Hi David!}  
3 Terra {Hi Terra!}  
4 NA    {Hi NA!}
```

str_flatten()

A função `str_flatten()` é útil para juntar textos em uma única saída de string.

```
> str_flatten(c("x", "y", "z"))  
[1] "xyz"
```

```
> str_flatten(c("x", "y", "z"), ", ")  
[1] "x, y, z"
```

```
> str_flatten(c("x", "y", "z"), ", ", last = ",  
  and ")  
[1] "x, y, and z"
```

str_flatten()

A função `str_flatten()` interage bem com a função `summarize()`

```
> df <- tribble(
  ~ name, ~ fruit,
  "Carmen", "banana",
  "Carmen", "apple",
  "Marvin", "nectarine",
  "Terence", "cantaloupe",
  "Terence", "papaya",
  "Terence", "mandarin"
)

> df |>
  group_by(name) |>
  summarize(fruits = str_flatten(fruit, ", "))
# A tibble: 3 × 2
  name      fruits
<chr>    <chr>
1 Carmen  banana, apple
2 Marvin    nectarine
3 Terence cantaloupe, papaya, mandarin
```

Separando variáveis em um único string

No pacote `{tidyr}` temos algumas funções que podem nos ajudar a separar as strings rapidamente

```
df |> separate_longer_delim(col, delim)
df |> separate_longer_position(col, width)
df |> separate_wider_delim(col, delim, names)
df |> separate_wider_position(col, widths)
```

Separando em linhas

Separar em linhas é mais direto.

```
df1 <- tibble(x = c("a,b,c", "d,e", "f"))  
df1 |>  
  separate_longer_delim(x, delim = ",")
```

```
# A tibble: 6 × 1
```

```
> x
```

```
  <chr>
```

1	a
2	b
3	c
4	d
5	e
6	f

Separando em colunas

Separar em colunas exige que a gente nomeie as novas colunas

```
> df3 <- tibble(x = c("a10.1.2022", "b10.2.2011", "e15
  .1.2015"))
> df3 |>
  separate_wider_delim(
    x,
    delim = ".",
    names = c("code", "edition", "year")
  )
# A tibble: 3 × 3
   code edition year
<chr> <chr>   <chr>
1 a10     1     2022
2 b10     2     2011
3 e15     1     2015
```

Separando em colunas

Podemos também por um NA como nome de uma variável para omitir uma parte da informação separada.

```
> df3 |>
  separate_wider_delim(
    x, delim = ".",
    names = c("code", NA, "year")
  )
# A tibble: 3 × 2
   code  year
<chr> <chr>
1 a10   2022
2 b10   2011
3 e15   2015
```

Separando em colunas

Podemos também usar `separate_wider_position()` para determinar a quantidade de caracteres em cada nova entrada.

```
> df4 <- tibble(x = c("202215TX", "202122LA", "202325CA"))
> df4 |>
  separate_wider_position(
    x,
    widths = c(year = 4, age = 2, state = 2)
  )
# A tibble: 3 x 3
  year   age state
<chr> <chr> <chr>
1 2022   15   TX
2 2021   22   LA
3 2023   25   CA
```

Separando em colunas

Se tiver um problema com a quantidade de elementos separados pelo delimitador, podemos ter um erro.

```
> df <- tibble(x = c("1-1-1", "1-1-2", "1-3", "1-3-2", "1"))
> df |>
  separate_wider_delim(
    x,
    delim = "-",
    names = c("x", "y", "z")
  )
```

```
Error in `separate_wider_delim()` :
! Expected 3 pieces in each element of `x`.
! 2 values were too short.
  Use `too_few = "debug"` to diagnose the problem.
  Use `too_few = "align_start"/"align_end"` to silence
  this message.
```

A própria mensagem de erro nos dá uma forma de verificaro isso. Podemos utilizar o arguemnto `too_few = "debug"` para investigar.

Separando em colunas

```
debug <- df |>
  separate_wider_delim(
    x,
    delim = "-",
    names = c("x", "y", "z"),
    too_few = "debug"
  )
# A tibble: 5 x 6
   x      y      z    x_ok x_pieces x_remainder
  <chr> <chr> <chr> <lgl>    <int>    <chr>
1 1-1-1 1      1    TRUE         3    ""
2 1-1-2 1      2    TRUE         3    ""
3 1-3   3     NA   FALSE         2    ""
4 1-3-2 3      2    TRUE         3    ""
5 1     NA     NA   FALSE         1    ""
```

Podemos filtrar os problemas, se a tabela for grande

```
debug |> filter(!x_ok)
```

`too_few = "align_start"` e `too_few = "align_end"` ajudam a preencher com NA se for o caso.

Separando em colunas

Podemos ter valores a mais também

```
df <- tibble(x = c("1-1-1", "1-1-2", "1-3-5-6", "1-3-2", "1-3-5-7-9"))
debug <- df |>
  separate_wider_delim(
    x,
    delim = "-",
    names = c("x", "y", "z"),
    too_many = "debug"
  )
# A tibble: 5 x 6
```

	x	y	z	x_ok	x_pieces	x_remainder
	<chr>	<chr>	<chr>	<lgl>	<int>	<chr>
1	1-1-1	1	1	TRUE	3	""
2	1-1-2	1	2	TRUE	3	""
3	1-3-5-6	3	5	FALSE	4	"-6"
4	1-3-2	3	2	TRUE	3	""
5	1-3-5-7-9	3	5	FALSE	5	"-7-9"

Aqui a coluna `x_remainder` nos mostra os excessos. Podemos utilizar `too_many = "drop"` ou `too_many = "merge"` para jogar fora ou juntar o que sobrou.

str_length()

Podemos verificar o comprimento de um string com a função `str_length()`

```
> str_length(c("a", "R for data science", NA))  
[1]  1 18 NA
```

Podemos combinar isso com outras funções

```
> babynames |>  
  count(length = str_length(name), wt = n)  
# A tibble: 14 × 2  
  length      n  
  <int>   <int>  
1     2 338150  
2     3 8589596  
3     4 48506739  
4     5 87011607  
5     6 90749404  
6     7 72120767  
# with 8 more rows
```

str_length()

```
> babynames |>
  filter(str_length(name) == 15) |>
  count(name, wt = n, sort = TRUE)
# A tibble: 34 × 2
  name                n
  <chr>              <int>
1 Franciscojavier    123
2 Christopherjohn    118
3 Johnchristopher    118
4 Christopherjame     108
5 Christophermich     52
6 Ryanchristopher     45
#       28 more rows
```

str_sub()

Podemos extrair parte do string com a função `str_sub()`

```
> str_sub(x, 1, 3)
[1] "App" "Ban" "Pea"
```

```
> str_sub(x, -3, -1)
[1] "ple" "ana" "ear"
```

E combinar ela com `mutate()` por exemplo.

```
> babynames |>
  mutate(
    first = str_sub(name, 1, 1),
    last = str_sub(name, -1, -1)
  )
# A tibble: 1,924,665 × 7
   year sex  name          n    prop first last
  <dbl> <chr> <chr>        <int> <dbl> <chr> <chr>
1  1880 F    Mary          7065 0.0724 M     y
2  1880 F    Anna          2604 0.0267 A     a
3  1880 F    Emma          2003 0.0205 E     a
4  1880 F    Elizabeth    1939 0.0199 E     h
# 1,924,661 more rows
```

Encoding

O padrão de encoding do `{readr}` é UTF-8 (Unicode Transformation Format – 8-bit)

Se os dados foram produzidos num formato mais antigo, podemos ter problemas

Lembrem que temos a função `guess_encoding()` do pacote `{readr}`, que pode nos ajudar

Alguns detalhes extras sobre encoding podem ser encontrados aqui.

Encoding

Como símbolos iguais podem ter a mesma aparência mas ser tratado de forma diferente dependendo da codificação, o `{readr}` nos dá algumas ferramentas para verificar a partir das funções `str_equal()` e `str_length()`

```
> u <- c("\u00fc", "u\u0308")
> str_view(u)
[1] | ü
[2] | u

> str_length(u)
[1] 1 2
> str_sub(u, 1, 1)
[1] "ü" "u"

> u[[1]] == u[[2]]
[1] FALSE
> str_equal(u[[1]], u[[2]])
[1] TRUE
```

Text

Podemos utilizar `str_view()` para verificar padrões

```
> str_view(fruit, "berry")  
[6] | bil<berry>  
[7] | black<berry>  
[10] | blue<berry>  
[11] | boysen<berry>  
[19] | cloud<berry>  
[21] | cran<berry>  
[29] | elder<berry>  
[32] | goji <berry>  
[33] | goose<berry>  
[38] | huckle<berry>  
[50] | mul<berry>  
[70] | rasp<berry>  
[73] | salal <berry>  
[76] | straw<berry>
```

Text

- ▶ Se utilizarmos ".", será entendido como qualquer caracter, e isso pode ser utilizado para encontrar combinações diferentes

```
> str_view(fruit, "a...e")  
[1] | <apple>  
[7] | bl<ackbe>rry  
[48] | mand<arine>  
[51] | nect<arine>  
[62] | pine<apple>  
[64] | pomegr<anate>  
[70] | r<aspbe>rry  
[73] | sal<al be>rry
```

Text

- ▶ ? torna um padrão opcional
- ▶ + deixa o padrão se repetir
- ▶ * deixa o padrão ser opcional ou ser repetir
- ▶ [] vai considerar um conjunto de caracteres

```
> str_view(words, "[aeiou]x[aeiou]")
[284] | <exa>ct
[285] | <exa>mple
[288] | <exe>rcise
[289] | <exi>st
> str_view(words, "[^aeiou]y[^aeiou]")
[836] | <sys>tem
[901] | <typ>e
```

Text

Podemos usar | para alternar padrões.

```
> str_view(fruit, "apple|melon|nut")
[1] | <apple>
[13] | canary <melon>
[20] | coco<nut>
[52] | <nut>
[62] | pine<apple>
[72] | rock <melon>
[80] | water<melon>
> str_view(fruit, "aa|ee|ii|oo|uu")
[9] | bl<oo>d orange
[33] | g<oo>seberry
[47] | lych<ee>
[66] | purple mangost<ee>n
```

str_detect()

A função `str_detect()` retorna um valor lógico ao verificar se um elemento do vetor combina com nosso conjunto.

```
> str_detect(c("a", "b", "c"), "[aeiou]")  
[1]  TRUE FALSE FALSE
```

str_detect()

A função `str_detect()` retorna um valor lógico ao verificar se um elemento do vetor combina com nosso conjunto.

```
> str_detect(c("a", "b", "c"), "[aeiou]")  
[1] TRUE FALSE FALSE
```

Isso pode ser útil para filtrar informação.

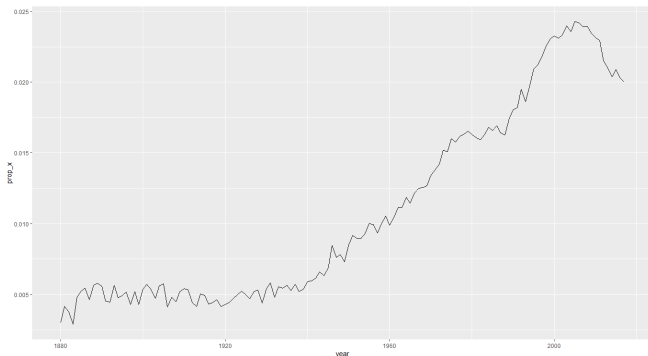
```
> babynames |>  
  filter(str_detect(name, "x")) |>  
  count(name, wt = n, sort = TRUE)
```

```
# A tibble: 974 × 2  
  name          n  
  <chr>      <int>  
1 Alexander  665492  
2 Alexis    399551  
3 Alex      278705  
4 Alexandra 232223  
5 Max       148787  
6 Alexa     123032  
# 968 more rows
```

str_detect()

Podemos utilizar isso para fazer algumas análises interessantes.

```
> babynames |>  
  group_by(year) |>  
  summarize(prop_x = mean(str_detect(name, "  
    ana")))) |>  
  ggplot(aes(x = year, y = prop_x)) +  
  geom_line()
```



str_replace() and str_replace_all()

Podemos utilizar essas funções para modificar os primeiros matches ou todos os matches.

```
x <- c("apple", "pear", "banana")  
> str_replace(x, "[aeiou]", "-")  
[1] "-pple"  "p-ar"   "b-nana"
```

```
x <- c("apple", "pear", "banana")  
> str_replace_all(x, "[aeiou]", "-")  
[1] "-ppl-"  "p--r"   "b-n-n-"
```

separate_wider_regex()

Podemos utilizar a função `separate_wider_regex()` para extrair variáveis que estejam juntas no mesmo string através de padrões.

```
> df
# A tibble: 7 × 1
  str
<chr>
1 <Sheryl>-F_34
2 <Kisha>-F_45
3 <Brandon>-N_33
4 <Sharon>-F_38
5 <Penny>-F_58
6 <Justin>-M_41
7 <Patricia>-F_84
```

separate_wider_regex()

```
> df |>
  separate_wider_regex(
    str,
    patterns = c(
      "<",
      name = "[A-Za-z]+",
      ">-",
      gender = ".", "_",
      age = "[0-9]+"
    )
  )
```

A tibble: 7 × 3

	name <chr>	gender <chr>	age <chr>
1	Sheryl	F	34
2	Kisha	F	45
3	Brandon	N	33
4	Sharon	F	38
5	Penny	F	58
6	Justin	M	41

Controlando padrões

Podemos utilizar a função `regex()` para controlar padrões, como caracteres maiúsculos e minúsculos.

```
> str_view(bananas, "banana")  
[1] | <banana>
```

```
> str_view(bananas, regex("banana", ignore_case  
  = TRUE))  
[1] | <banana>  
[2] | <Banana>  
[3] | <BANANA>
```

Outras funções

- ▶ `str_trim()` remove espaços no início e no final, enquanto `str_squish()` remove espaços extras no texto
- ▶ `str_to_lower()`, `str_to_upper()` e `str_to_title()` trocam para minúscula, maiúscula e primeira inicial maiúscula
- ▶ `str_extract()`, `str_extract_all()`, `str_match()` e `str_split()` ajudam a extrair conteúdo dos textos

Referências I

-  LITTLE, Roderick JA; RUBIN, Donald B. **Statistical analysis with missing data**. [S. l.]: John Wiley & Sons, 2019.
-  VAN BUUREN, Stef. **Flexible imputation of missing data**. [S. l.]: CRC press Boca Raton, FL, 2012. v. 10.